

RETO 2 — PATRONES DE DISEÑO ARQUITECTÓNICO

María Alexandra Jiménez Suarez

Juan José Álvarez Restrepo

Carolina Ramírez Lotero

Natalia Giraldo Morales

Universidad Pontificia Bolivariana, Facultad de Ingeniería

Arquitectura de Software

Cesar Augusto López Gallego

30 de agosto de 2026

ACTIVIDAD 1	2
1.1 Análisis de puntos de dolor.	2
ACTIVIDAD 2	4
Entregable 2.1: tabla de decisión de patrones.	4
Entregable 2.2: bitácora de decisiones frente a la IA.	6
ACTIVIDAD 3	8
Entregable 3.1.: diagramas.	8
Entregable 3.2.: tabla de cambio estructural.	8
Entregable 3.3.: ficha por cada patrón adoptado.	12
ACTIVIDAD 4	15
Entregable 4.1.: matriz de verificación.	15
Entregable 4.2.: Evidencias de que el comportamiento no cambia.	16
ACTIVIDAD 5	17
Entregable 5.1.: registro de riesgos.	17
ACTIVIDAD 6	18

ACTIVIDAD 1

1.1 Análisis de puntos de dolor.

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad
P-01	Program.cs	Cada evento, repositorio y servicio se instancia a mano con new, en orden fijo, en el mismo archivo que además contiene login y menú. Es el único lugar del sistema donde se puede leer cómo se arma el grafo de objetos completo.	Para 1 sola familia nueva el peso son 2 archivos nuevos + 2 modificados.	Alta
P-02	ProductoFactory.cs	La fábrica conoce directamente las implementaciones concretas que puede crear y solo contempla las variantes existentes. Si aparecen nuevas variantes de producto que deban construirse mediante ella, la decisión de qué clase instanciar queda concentrada en este punto.	1 archivo + 1 método/ramificación adicional por nueva variante que se incorpore mediante la fábrica.	Alta
P-03	ServicioProducto.cs	La lógica de carga está relacionada con la forma concreta en que se obtienen y construyen los productos. Cuando aparece una nueva variante, el mecanismo de carga requiere modificar o agregar una conexión específica para incorporarla al servicio.	1 clase + 1 punto de conexión por nueva variante.	Alta
P-04	Jerarquía Producto → Medicamento → MedicamentoCapsula / MedicamentoLiquido	La jerarquía permite trabajar con los subtipos a través de Producto y no presenta actualmente un costo de evolución suficientemente alto que justifique introducir otro patrón.	Introducir un patrón sobre esta jerarquía agregaría clases e indirección sin resolver un problema demostrado. Se conserva porque actualmente cumple su propósito y presenta bajo riesgo.	No intervenir
P-05	ServicioDescuento.cs Y IDescuento.cs	Existe la interfaz y una única implementación que devuelve siempre 10% fijo. Ningún servicio la recibe por inyección y Program.cs no la instancia: no participa en ningún flujo real de venta. Es una extensión ya prevista que quedó sin conectar. En resumen, es una abstracción preparada para variar la política de descuento, pero no existe un punto de integración que permita utilizarla en el flujo.	2 archivos existen, 0 puntos de uso (confirmado por búsqueda). Si el negocio pide una segunda regla de descuento, no hay dónde conectarla porque nada la consume hoy.	Media

P-06	ServicioNotificacion.cs Y IServicioNotificacion.cs	Existe una interfaz y una implementación genérica de notificación (EnviarNotificacion) que imprime en consola) que no se relaciona con ninguno de los 4 notificadores específicos (EventoStockMinimo, etc.). Confirmamos por búsqueda que ningún servicio ni Program.cs la instancia ni la inyecta, es un segundo mecanismo de notificación, paralelo y no usado, que convive con el que sí funciona	2 archivos existen, 0 puntos de uso. Antes de decidir cómo notificar algo nuevo, hay que decidir primero cuál de los dos mecanismos de notificación que ya existen en el proyecto es el que se debe extender.	Baja
P-07	RepositorioMedicamentoArchivo.cs	Al crear el Laboratorio desde productos.txt, solo se lee el nombre . La ciudad y el teléfono quedan fijos en código como "Medellin" y "4444444". Laboratorio.cs define nombre, dirección y teléfono como datos del laboratorio, pero estos no se obtienen del archivo.	requiere modificar RepositorioMedicamentoArchivo.cs, ampliar el formato de productos.txt y coordinar con quien genera el archivo.	Media

ACTIVIDAD 2

Entregable 2.1: tabla de decisión de patrones.

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
Factory Method	Creacional	P-02 (ProductoFactory concentra la decisión y solo contempla variantes existentes)	Gana: por que centraliza la creación, facilita la extensión siguiendo Open/Closed y ayuda a desacoplar las clases de producto concretas. Cuesta: agrega clases/métodos de creación y puede aumentar la estructura del código.	Adoptado	Ataca directamente la rigidez identificada en P-02 y facilita la incorporación de nuevas variantes sin modificar continuamente una única fábrica central.
Abstract Factory	Creacional	P-01 (Program.cs instancia manualmente todo el grafo de objetos).	Gana: reduce el conocimiento de implementaciones concretas en Program.cs y centraliza la creación. Cuesta: introduce más abstracciones y solo aporta valor si existen familias de objetos relacionados.	Descartado	El problema actual requiere desacoplar la creación, pero no existe evidencia suficiente de múltiples familias de objetos que deban crearse conjuntamente. Factory Method resulta más proporcional.

Builder	Creacional	P-01 (candidato descartado tras revisar el deck)	Se exige director + Builder concreto para la construcción paso a paso donde el proceso cambia según el tipo de producto. Cuando NO usarlo: en la construcción de objetos simples: no usar para objetos con pocos parámetros, ni cuando se quiere evitar introducir mayor complejidad.	Descartado	Ninguno de nuestros constructores califica como "constructor telescópico" difícil de manejar, y el proceso de ensamblar Program.cs no varía según tipo de producto, es wiring de dependencias, no construcción de un producto complejo. Se resuelve mejor con Facade.
Adapter	Estructural	P-06 – P-07 (RepositorioMedicamentoArchivo depende de un formato específico de datos del archivo).	Gana: permitiría adaptar una fuente de datos con una interfaz diferente sin modificar directamente el consumidor. Cuesta: agrega una capa de adaptación.	Descartado	P-07 corresponde principalmente a datos incompletos o fijados en código, no a una incompatibilidad entre interfaces. Adapter no resuelve directamente el problema encontrado.
Facade	Estructural	P-01 (Program.cs arma todo a mano, mezcla ensamblaje con login y menú)	Gana: un único punto de entrada simple para todo lo que hoy hay que leer completo para entender cómo se arma el sistema. Cuesta: la fachada termina conociendo todos los subsistemas internos, así que, si uno cambia, hay que actualizarla.	Adoptado	Es el mapeo directo de P-01: hoy Program.cs es el único lugar donde se puede leer cómo se arma el grafo completo de objetos. La fachada recibe sus dependencias por inyección (no las crea con new dentro de sí misma), evitando repetir el mismo problema en otro archivo.
Decorator	Estructural	P-05 (ServicioDescuento existe, pero está desconectado del flujo real).	Gana: permite agregar responsabilidades al servicio de descuento sin modificar directamente su implementación base. Cuesta: agrega clases y capas adicionales que pueden aumentar la complejidad.	Descartado	El problema de P-05 no es la necesidad de agregar responsabilidades sobre un mismo objeto, sino que el servicio de descuento actualmente no está conectado al flujo de venta. Decorator no resuelve directamente esta rigidez.

Strategy	Comportamiento	P-05 (ServicioDescuento tiene una única regla fija del 10 %).	Gana: permite encapsular diferentes reglas de descuento y agregar nuevas sin modificar una lógica central. Cuesta: requiere una estrategia por regla y un mecanismo para seleccionarla.	Adoptado	Es adecuado para SC-3 si se requieren diferentes reglas de descuento según el convenio. Convierte cada regla en un comportamiento independiente y extensible.
Template Method	Comportamiento	P-03 (los métodos de carga en ServicioProducto se repiten casi igual)	Gana: fijar un esqueleto común de carga que cada variante solo completa en su parte distinta. Cuesta: convertir una relación de composición (IRepositorioProducto inyectado) en una de herencia.	Descartado	Al resolver P-01 con Facade y P-02 con Factory Method, la duplicación de P-03 se resuelve como consecuencia: la fachada decide qué repositorio usar, y ya no hacen falta dos métodos casi iguales en ServicioProducto. No amerita un patrón propio.
State	Comportamiento	P-05 si el crédito requiere comportamientos según su estado.	Gana: permite cambiar el comportamiento de un objeto según su estado. Cuesta: agrega clases para representar los estados y sus transiciones.	Descartado	Los puntos de dolor actuales no evidencian estados con comportamientos diferentes. Introducir State únicamente por la existencia del concepto de crédito sería sobreingeniería.

Entregable 2.2: bitácora de decisiones frente a la IA.

Criterio de validación de las propuestas

Las consultas realizadas a la herramienta de IA se utilizaron como apoyo para explorar alternativas de diseño y contrastar las decisiones del equipo, pero no como sustituto del análisis del código. Cada propuesta fue revisada frente a las clases, archivos y relaciones existentes en el sistema, verificando si el problema identificado realmente estaba presente y si el patrón sugerido respondía a dicho problema. De esta manera, una propuesta de la IA podía ser aceptada, descartada o ajustada según la evidencia encontrada en el código.

La revisión también permitió identificar situaciones en las que introducir un patrón no aportaría un beneficio proporcional a su costo. En particular, se decidió no incorporar patrones únicamente para aumentar la cantidad de soluciones aplicadas, sino conservar

aquellas decisiones que respondieran a un punto de dolor concreto y demostrable. Este criterio permitió mantener intervenciones acotadas y evitar agregar clases, relaciones o niveles de indirección que no fueran necesarios para la evolución planteada.

ID	Qué consultaron	Qué propuso la herramienta	Qué hicieron	Argumento propio y evidencia
B-01	Qué patrón resolvía P-01 (Program.cs arma todo a mano)	Propuso combinar Builder + Facade	Corregimos: descartamos Builder, dejamos solo Facade	Revisamos los constructores que existen actualmente en el proyecto y ninguno supera los 8 parámetros. Además, el proceso de armar Program.cs no cambia según el tipo de producto. Por esto consideramos que Builder no responde al problema identificado, ya que está orientado a construcciones paso a paso que pueden variar, mientras que aquí el problema corresponde al ensamblaje de dependencias.
B-02	Qué patrón resolvía P-02 (ProductoFactory cerrado a variantes)	Propuso Factory Method, con Abstract Factory como alternativa	Aceptamos Factory Method, rechazamos Abstract Factory	Revisamos Cosmetico.cs y Comestible.cs y no encontramos una variación coordinada con Medicamento que requiera crear varios objetos relacionados como una familia. Por esta razón, consideramos que Abstract Factory introduciría una estructura adicional y compleja sin responder a una necesidad presente en el código.
B-03	Qué patrón resolvía P-06/P-07 (notificación paralela y laboratorio hardcodeado)	Propuso Adapter	Rechazamos	Revisamos RepositorioMedicamentoArchivo.cs directamente para comprobar de dónde venía el problema identificado. Encontramos que los datos de ciudad y teléfono están fijados en el código ("Medellin", "4444444"), pero no existe una incompatibilidad entre interfaces. Por esto descartamos Adapter, ya que el problema corresponde a la forma en que se manejan los datos y no a la necesidad de adaptar contratos incompatibles.
B-04	Si Strategy encajaba mejor que Decorator para P-05 y SC-3	Aceptó el cambio y lo justificó con la interfaz ya existente	Fue idea nuestra proponer el cambio; aceptamos la justificación final	Leímos IDescuento.cs y ServicioDescuento.cs. La interfaz ya existe, actualmente hay una sola implementación con un descuento fijo del 10% y confirmamos mediante búsqueda que no tiene puntos de uso. Consideramos que Strategy se ajusta mejor porque permite representar distintas reglas de descuento sin modificar directamente el flujo que las utiliza. La decisión parte de la estructura actual del código y no únicamente de la definición del patrón.
B-05	Si Template Method resolvía la duplicación de P-03	Lo propuso inicialmente como adopción	Corregimos: lo descartamos	Argumentamos que si P-01 se resuelve con Facade y P-02 con Factory Method, la fachada terminaría decidiendo qué repositorio usar, la duplicación en ServicioProducto desaparece como efecto colateral. Forzar herencia sobre algo que hoy funciona por composición (IRepositorioProducto inyectado) no se justifica.
B-06	Si State aplicaba al "crédito descontable" de SC-3	Fue idea nuestra consultarlo — no lo había sugerido la IA antes	La IA lo evaluó y coincidió en descartarlo; aceptamos el descarte	Revisamos Cliente.cs para comprobar si existía realmente un comportamiento relacionado con estados de crédito. Actualmente la clase maneja Puntos, pero el concepto de crédito todavía no está modelado. Como no encontramos un

				comportamiento que cambie según un estado interno del objeto, consideramos que incorporar State en este momento sería anticiparse a un problema que el código actual no presenta.
B-07	Si Bridge aplicaba a la jerarquía Medicamento → Cápsula/Líquido (P-04)	Lo evaluó y lo descartó por su cuenta	Aceptamos, pero verificamos antes de dar por cerrado	Revisamos las variaciones existentes en la jerarquía de productos y encontramos una sola dimensión principal: el tipo de medicamento. No identificamos dos dimensiones independientes que deban evolucionar y combinarse entre sí. Por esto consideramos que Bridge no aporta un beneficio suficiente para la situación actual y mantenemos la decisión de no intervenir P-04.
B-08	Pedimos una segunda pasada del cuadro completo, alineada a las diapositivas del profesor	Reescribió las justificaciones citando frases textuales del deck	Corregimos: pedimos que no sonara a copiar diapositivas, sino a explicación propia	Consideramos importante revisar las decisiones con el material de clase y no limitar el análisis a la respuesta inicial de la IA. La formulación debía poder ser comprendida y defendida por el grupo durante la sustentación, por lo que contrastamos las propuestas con los conceptos trabajados y ajustamos la redacción cuando fue necesario.
B-09	Verificamos si de verdad RegistrarVenta/IDescuento no tienen ningún uso, antes de aceptar esa premisa	Afirmó "cero puntos de uso"	Aceptamos, pero solo tras comprobarlo nosotros con grep en todo el repositorio	No tomamos como suficiente la afirmación de la IA sobre la ausencia de usos de ServicioDescuento e IDescuento. Realizamos una búsqueda sobre el proyecto y confirmamos que no aparecen invocaciones fuera de sus propias definiciones. Esto nos permitió respaldar la decisión con una comprobación directa del código.
B-10	Si convenía forzar un cuarto patrón adoptado, ya que el rango permite hasta 5	No propuso ningún cuarto patrón	Fue idea nuestra evaluar si faltaba algo; decidimos no forzarlo	Cada patrón adoptado debe estar anclado a evidencia real, no al cupo disponible. Si no hay un cuarto punto de dolor que lo justifique, quedarse en 3 es la decisión correcta, no una carencia.
B-11	Antes de aceptar que P-07 "es un problema de datos y no de patrón"	Volvió a citar el fragmento exacto de RepositorioMedicamentoArchivo.cs con los valores hardcodeados	Aceptamos, y dejamos anotado en el documento que se resuelve ampliando el formato de productos.txt, no con un patrón	Antes de cerrar P-07 como un punto que no requería un patrón, revisamos directamente la implementación y los datos involucrados. Confirmamos que el problema está relacionado con valores de laboratorio fijados en el código y con la información disponible en productos.txt, no con una incompatibilidad entre componentes. Por esto consideramos que la solución corresponde a una mejora en el manejo de los datos y no a la incorporación de Adapter.

ACTIVIDAD 3

Entregable 3.1.: diagramas.

Primer diagrama y Segundo diagrama: ambos diagramas fueron subidos al repositorio presente en GitHub para un mejor formato y calidad.

Entregable 3.2.: tabla de cambio estructural.

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía de él y cómo se reconecta
E-01	Program.cs	Se transforma	Concentraba el ensamblaje de objetos, la carga de información, el login, las consultas, la venta, los puntos y la interacción del menú. Además, accedía directamente a los servicios para ejecutar cada operación.	Se mantiene como composition root, pero delega las operaciones del sistema en FarmaciaFachada. Allí se crean las dependencias concretas, se configuran los eventos y se construye el grafo de objetos.	Antes el usuario de consola dependía directamente de ServicioProducto, ServicioCliente, ServicioUsuario y ServicioMovimiento. Ahora la interacción se reconecta principalmente mediante FarmaciaFachada, que coordina los servicios internos.
E-02	FarmaciaFachada	Entra	No existía un punto único para acceder a las operaciones principales del sistema. Program.cs conocía directamente varios servicios.	Centraliza el acceso a las funcionalidades principales: carga de datos, autenticación, alertas, consultas, ventas y puntos. Recibe los servicios mediante constructor.	Program.cs pasa a depender de la fachada para ejecutar las operaciones. La fachada, a su vez, se conecta con ServicioProducto, ServicioCliente, ServicioUsuario, ServicioVenta, ServicioAutenticacion y ServicioAtencionClinica.
E-03	FabricaProducto	Entra	No existía una abstracción común para las fábricas. ProductoFactory construía directamente los tipos concretos.	Define el contrato CrearProducto (string [] datos) para las fábricas concretas.	Los repositorios de productos reciben una FabricaProducto, permitiendo que la construcción concreta quede separada de la lectura del archivo.
E-04	FabricaMedicamentoCapsula	Entra	La creación de MedicamentoCapsula estaba directamente dentro de RepositorioMedicamentoArchivo y, adicionalmente, ProductoFactory también conocía su construcción.	Se encarga exclusivamente de construir medicamentos cápsula a partir de los datos recibidos.	RepositorioMedicamentoArchivo delega la construcción en esta fábrica mediante FabricaProducto.
E-05	FabricaMedicamentoLiquido	Entra	No existía una fábrica específica para la construcción desde archivo.	Encapsula la creación de MedicamentoLiquido, incluyendo laboratorio, material de envase y volumen.	Puede ser utilizada como implementación de FabricaProducto cuando el repositorio necesite construir medicamentos líquidos.
E-06	FabricaCosmetico	Entra	RepositorioCosmeticoArchivo creaba directamente cada Cosmético.	Centraliza la construcción de objetos Cosmético.	RepositorioCosmeticoArchivo recibe una FabricaProducto y delega en ella la creación.
E-07	FabricaComestible	Entra	RepositorioComestibleArchivo construía directamente cada Comestible.	Centraliza la construcción de objetos Comestible.	RepositorioComestibleArchivo delega la creación en esta fábrica a través de FabricaProducto.
E-08	ProductoFactory	Se transforma	Construía directamente MedicamentoCapsula y	Se mantiene como una fachada estática sobre	Los consumidores que utilicen ProductoFactory siguen

			MedicamentoLiquido, con valores como stock mínimo, fechas, relleno y envase definidos internamente.	las fábricas concretas y delega CrearCapsula () y CrearLiquido () en FabricaMedicamentoCapsula y FabricaMedicamentoLiquido.	accediendo mediante sus métodos públicos, pero la construcción real queda delegada en las fábricas concretas. En la carga desde archivos, los repositorios ya no pasan por ProductoFactory.
E-09	RepositorioMedicamentoArchivo	Se transforma	Leía el archivo y construía directamente MedicamentoCapsula, incluyendo la creación de Laboratorio.	Continúa leyendo el archivo, pero delega la creación del producto en una instancia de FabricaProducto, por defecto FabricaMedicamentoCapsula.	ServicioProducto sigue dependiendo de IRepositorioProducto; ahora el repositorio reconecta la lectura de datos con la fábrica en lugar de construir directamente el objeto.
E-10	RepositorioCosmeticoArchivo	Se transforma	Leía el archivo y creaba directamente cada Cosmético.	Lee los datos y delega la construcción en FabricaCosmetico mediante FabricaProducto	ServicioProducto continúa dependiendo únicamente de IRepositorioProducto; el repositorio absorbe el detalle de persistencia y delega la creación.
E-11	RepositorioComestibleArchivo	Se transforma	Leía el archivo y construía directamente cada Comestible.	Comestible. Lee los datos y delega la creación en FabricaComestible.	ServicioProducto continúa utilizando IRepositorioProducto; la creación concreta queda dentro de la fábrica.
E-12	IVendible	Entra	No existía una abstracción común para representar aquello que puede venderse. Movimiento solo aceptaba Producto.	Define el contrato común de un elemento vendible: Nombre, Precio y MostrarInformacion ().	Producto y ServicioClinico implementan IVendible. ServicioVenta y Movimiento pueden trabajar con ambos tipos sin exigir que sean productos farmacéuticos.
E-13	Producto	Se transforma	Era la clase base de los productos y no implementaba una abstracción para otros elementos vendibles.	Mantiene sus atributos y comportamiento existentes, pero ahora implementa IVendible.	Las operaciones que antes recibían específicamente Producto pueden utilizar IVendible cuando necesitan trabajar con cualquier elemento vendible.
E-14	Movimiento	Se transforma	Tenía una referencia directa a Producto, por lo que un movimiento solamente podía asociarse a un producto.	Mantiene compatibilidad con productos, pero agrega IVendible Item y un constructor que recibe cualquier elemento vendible. Conserva además la propiedad Producto para acceder al producto cuando corresponda.	ServicioMovimiento continúa registrando movimientos. ServicioVenta puede crear movimientos tanto para productos como para servicios clínicos.
E-15	ServicioClinico	Entra	No existía una representación de los servicios ofrecidos por la farmacia.	Representa un servicio vendible con nombre, precio y descripción. Implementa IVendible.	ServicioAtencionClinica administra estos objetos y FarmaciaFachada los expone para búsqueda y consulta.

					ServicioVenta puede registrar su venta mediante IVendible.
E-16	FabricaServicio / FabricaServicio Clinico	Entra	No existía una fábrica para servicios clínicos.	Define y concreta la creación de ServicioClinico a partir de los datos del archivo o de parámetros directos.	RepositorioServicioArchivo utiliza FabricaServicio para construir los servicios sin realizar directamente la creación dentro del repositorio.
E-17	IRepositorioServicioClinico	Entra	No existía una abstracción para la fuente de datos de servicios clínicos.	Define las operaciones para verificar la existencia del archivo y leer los servicios.	ServicioAtencionClinica depende de esta interfaz, mientras RepositorioServicioArchivo proporciona la implementación concreta.
E-18	RepositorioServicioArchivo	Entra	No existía un repositorio para los servicios clínicos.	Lee servicios.txt y delega la construcción de cada servicio en FabricaServicio.	ServicioAtencionClinica lo recibe mediante IRepositorioServicioClinico, evitando depender directamente del archivo.
E-19	ServicioAtencionClinica	Entra	No existía un servicio encargado de administrar servicios clínicos.	Mantiene la colección de servicios, permite agregarlos, consultarlos, buscarlos y cargarlos mediante IRepositorioServicioClinico.	FarmaciaFachada utiliza este servicio para consultar y buscar servicios. Program.cs accede a ellos mediante la fachada.
E-20	ServicioVenta	Entra	Program.cs realizaba directamente la disminución de stock y la creación del movimiento durante una venta. No existía un servicio que coordinara estas operaciones.	Centraliza el registro de una venta, coordina el descuento, el movimiento y el descuento de stock cuando el elemento vendido es un producto. También permite vender ServicioClinico.	FarmaciaFachada delega RegistrarVenta () en ServicioVenta. Este se reconecta con ServicioProducto, ServicioMovimiento e IDescuento.
E-21	IDescuento	Se transforma en punto de extensión	Existía como interfaz, pero no participaba realmente en el flujo de venta.	Se utiliza como abstracción de la estrategia de descuento que recibe ServicioVenta.	ServicioVenta depende de IDescuento, y Program.cs proporciona la implementación concreta al construir el servicio.
E-22	ServicioDescuento	Se transforma en uso	Implementaba un descuento fijo del 10 %, pero no estaba conectado al flujo real de venta.	Se utiliza como estrategia inicial de descuento para ServicioVenta.	Program.cs instancia ServicioDescuento como IDescuento y lo inyecta en ServicioVenta.
E-23	DescuentoSin Descuento	Entra	No existía una estrategia que representara explícitamente la ausencia de descuento.	Implementa IDescuento y retorna cero como descuento.	Puede sustituir la estrategia utilizada por ServicioVenta mediante EstablecerEstrategiaDescuento ().

E-24	DescuentoPorcentaje	Entra	La única regla disponible era el 10 % fijo de ServicioDescuento.	Permite calcular descuentos mediante un porcentaje configurable.	Implementa IDescuento y puede ser asignada a ServicioVenta sin modificar este servicio.
E-25	DescuentoMontoFijo	Entra	No existía una estrategia para descuentos de monto fijo.	Implementa IDescuento y calcula un descuento limitado al precio de la venta.	Puede ser suministrada a ServicioVenta como una estrategia alternativa.
E-26	ServicioProducto.DescontarStock()	Entra	Program.cs modificaba directamente producto.Stock -= cantidad durante la venta. No existía una operación centralizada para descontar stock y controlar stock insuficiente.	ServicioProducto incorpora DescontarStock (), valida la disponibilidad y modifica el stock desde el servicio encargado de los productos.	ServicioVenta deja de modificar directamente el producto y solicita el descuento de stock a ServicioProducto.
E-27	ServicioProducto.CargarDesdeArchivo()	Se transforma	Ya utilizaba IRepositoryProducto, pero tenía la lógica principal de carga directamente en el método y exponía dos variantes de carga.	El método principal delega en la sobrecarga que recibe el repositorio. Se mantiene la abstracción de repositorio y se evita duplicar la lógica de carga.	FarmaciaFachada utiliza ServicioProducto para cargar medicamentos, cosméticos y comestibles; los repositorios se encargan de las implementaciones concretas.
E-28	ServicioAutenticacion	Se consolida / entra en el flujo de fachada	Ya existía en la solución previa como separación de la autenticación de ServicioUsuario, pero Program.cs la utilizaba directamente.	Continúa coordinando el login, pero ahora es una dependencia interna de FarmaciaFachada.	Program.cs deja de invocarlo directamente y utiliza farmaciaFachada.Autenticar().
E-29	AspectoAutenticacion	Se transforma	Comparaba directamente usuario y contraseña mediante igualdad exacta.	Conserva la misma responsabilidad, pero incorpora validación de valores vacíos y limpieza de espacios y caracteres BOM antes de comparar credenciales.	ServicioAutenticacion continúa delegando en esta clase; no se cambia la ubicación de la responsabilidad de autenticación.
E-30	Program.cs + servicios.txt	Se transforma / Entra	El sistema no cargaba ni mostraba servicios clínicos porque no existían dentro del flujo.	Se incorpora la carga de servicios.txt y una nueva opción del menú para consultar los servicios clínicos.	Program.cs pasa por FarmaciaFachada, que delega la carga en ServicioAtencionClinica y su repositorio.

Entregable 3.3.: ficha por cada patrón adoptado.

Patrón 1 - Facade

Campo	Contenido
Patrón y punto de dolor que resuelve	Facade , responde a P-01 : Program.cs instanciaba con new cada evento, repositorio y servicio a mano, en orden fijo, en el mismo archivo que además contenía login y menú.

	Implementado en BibFarmacia/Fachada/FarmaciaFachada.cs (clase completa, línea 11 en adelante).
Alternativas que evaluaron	Las alternativas evaluadas según tabla 2.1: (a) No hacer nada : dejar a Program.cs como único punto de ensamblaje y consumo directo de ServicioProducto, ServicioCliente, ServicioUsuario, ServicioMovimiento. (b) Abstract Factory : descartada porque no existe evidencia de familias de objetos relacionados que deban crearse conjuntamente — solo hay un problema de ensamblaje, no de familias de creación. (c) Builder : descartada (B-01) porque ningún constructor supera 8 parámetros y el proceso de ensamblar no varía según tipo de producto; no es una "construcción paso a paso" que cambie. (d) Elegida : Facade.
Qué sale y qué entra	Salida : el acceso directo de Program.cs/consola a ServicioProducto, ServicioCliente, ServicioUsuario, ServicioMovimiento uno por uno (E-01). Entrada : FarmaciaFachada (E-02), que centraliza carga de datos, autenticación, alertas, consultas, ventas y puntos, recibiendo los servicios por constructor. Como parte de nuestro refactor reciente (corrección de P-01 más a fondo), también entra ServicioVenta (nuevo, BibFarmacia/Servicios/ServicioVenta.cs) porque RegistrarVenta (antes en la fachada, línea ~115) absorbía cálculo de descuento y mutación de Stock, algo que su propia regla de arquitectura (SRP) prohíbe para una fachada.
Cómo se relaciona	Program.cs sigue siendo el composition root: construye eventos, repositorios y servicios, y con ellos arma ServicioVenta y finalmente FarmaciaFachada (constructor, FarmaciaFachada.cs línea 11). El menú de consola solo llama métodos de la fachada. Internamente, la fachada delega en ServicioVenta.RegistrarVenta (línea 115-127), que a su vez consume el patrón Strategy (ficha 3, IDescuento) para calcular el descuento; es el punto exacto donde Facade y Strategy interactúan.
Impacto	Creadas : FarmaciaFachada.cs, y (extensión posterior) ServicioVenta.cs. Modificado : Program.cs (deja de invocar servicios directamente, pasa a invocar la fachada), ServicioProducto.cs (se agrega DescontarStock para que el inventario no lo mute un tercero). Eliminada : ninguna clase. Efecto en Anexo B: cualquier solicitud de cambio sobre el flujo de venta o sobre qué se ve en consola ahora se resuelve tocando FarmaciaFachada/ServicioVenta, sin tocar Program.cs.
Qué cuesta	La fachada termina conociendo todos los subsistemas internos (tal como advierte la tabla 2.1): si ServicioProducto o ServicioVenta cambian su forma de construirse, hay que actualizar la fachada. Añade una clase más de indirección entre la consola y la lógica real, lo que implica un salto extra al depurar un flujo de venta.
Origen	Adoptado por el equipo, con apoyo de la IA (bitácora B-01): la herramienta propuso inicialmente combinar Builder + Facade; el equipo corrigió la propuesta descartando Builder tras revisar que ningún constructor calificaba como "constructor telescópico". La extracción de ServicioVenta (para que la fachada dejara de calcular) fue propuesta por la herramienta en una sesión posterior de esta misma conversación y aceptada sin corrección por el equipo

Patrón 2 - Factory Method

Campo	Contenido
Patrón y punto de dolor que resuelve	Factory Method , responde a P-02 : ProductoFactory.cs conocía directamente las implementaciones concretas y solo contemplaba las variantes existentes, si aparecía una variante nueva, la decisión de qué clase instanciar quedaba concentrada ahí. Implementado en BibFarmacia/Factories/FabricaProducto.cs (clase abstracta, línea 5; método abstracto CrearProducto (string [] datos), línea 7), con subclases FabricaMedicamentoCapsula.cs, FabricaMedicamentoLiquido.cs, FabricaCosmetico.cs, FabricaComestible.cs.
Alternativas que evaluaron	(a) No hacer nada : mantener ProductoFactory como único punto de construcción, con el costo declarado en P-02 de "1 archivo + 1 método/ramificación adicional por variante nueva". (b) Abstract Factory : descartada (tabla 2.1) porque no hay evidencia de familias de objetos

	relacionados que deban crearse juntos, solo variantes independientes de Producto. (c) Elegida: Factory Method, con una fábrica concreta por tipo heredando de FabricaProducto.
Qué sale y qué entra	No sale ProductoFactory.cs, se transforma (E-08): en vez de construir MedicamentoCapsula/MedicamentoLiquido con sus propios valores internos, ahora es una fachada estática delgada que delega en FabricaMedicamentoCapsula.Crear(...) y FabricaMedicamentoLiquido.Crear(...) (ProductoFactory.cs líneas 14-27). Entran como nuevos participantes: FabricaProducto (abstracción, E-03), FabricaMedicamentoCapsula (E-04), FabricaMedicamentoLiquido (E-05), FabricaCosmetico (E-06), FabricaComestible (E-07), cada una con el papel de construir su tipo concreto a partir de string [] datos vía CrearProducto(...).
Cómo se relaciona	Cada Repositorio*Archivo recibe su FabricaProducto por constructor y la invoca en LeerProductos (línea 30: fabrica.CrearProducto(datos)) al parsear cada línea del .txt. Los objetos creados llegan a ServicioProducto/ServicioAtencionClinica (vía CargarDesdeArchivo), y de ahí a FarmaciaFachada.ObtenerProductos(). No interactúa en tiempo real con Facade ni Strategy — pera "aguas arriba", en la carga inicial, mientras que Strategy actúa en el momento de la venta.
Impacto	Creadas: FabricaProducto.cs, FabricaMedicamentoCapsula.cs, FabricaMedicamentoLiquido.cs, FabricaCosmetico.cs, FabricaComestible.cs. Modificado: ProductoFactory.cs (se le quita la construcción propia, pasa a delegar), cada Repositorio*Archivo (recibe la fábrica por constructor en vez de construir directamente, E-06/E-07). Eliminada: ninguna clase. Efecto Anexo B: agregar una nueva variante de producto ya no toca ProductoFactory ni el repositorio existente — solo se crea una subclase nueva de FabricaProducto.
Qué cuesta	Una clase de fábrica por cada tipo de producto (hoy 4, más ProductoFactory que queda como envoltorio). Hoy ProductoFactory.CrearCapsula/CrearLiquido no tiene ningún punto de uso real en el flujo (confirmado por búsqueda) — es una API preparada pero todavía sin consumidor, similar en espíritu a lo detectado en P-05/P-06 aunque con menor severidad porque sí quedó correctamente conectada como parte del Factory Method (los repositorios sí usan CrearProducto, solo Crear(...) queda sin llamador). Además, RepositorioMedicamentoArchivo sigue usando por defecto FabricaMedicamentoCapsula (línea 15) y nada en Program.cs inyecta FabricaMedicamentoLiquido, así que esa fábrica ya existe, pero no participa en la carga real de productos.txt.
Origen	Adoptado por el equipo con apoyo de la IA (bitácora B-02): la herramienta propuso Factory Method con Abstract Factory como alternativa; el equipo rechazó Abstract Factory tras revisar Cosmetico.cs y Comestible.cs y no encontrar variación coordinada que requiera crear varios objetos como familia.

Patrón 3 - Strategy

Campo	Contenido
Patrón y punto de dolor que resuelve	Strategy, responde a P-05 . En el sistema existía la interfaz IDescuento con una sola implementación (ServicioDescuento, descuento fijo del 10%), pero ningún servicio la recibía por inyección y Program.cs no la instanciaba: era una abstracción ya prevista en el diseño, pero sin ningún punto de conexión al flujo real de venta. El contrato CalcularDescuento (decimal precio) está definido en IDescuento.cs, y hoy cuenta con cuatro implementaciones concretas en BibFarmacia/EstrategiasDescuento/: ServicioDescuento (10% fijo), DescuentoSinDescuento (0%), DescuentoPorcentaje (porcentaje configurable) y DescuentoMontoFijo (monto tope, nunca supera el precio base).
Alternativas que evaluaron	(a) No hacer nada: resolver el descuento con un if dentro de ServicioVenta. Se descartó porque no resuelve P-05 y viola OCP en cuanto aparezca una segunda regla. (b) Decorator: el problema de P-05 no es agregar responsabilidades sobre un mismo objeto, sino que el servicio de descuento estaba desconectado del flujo de venta; Decorator no resuelve una

	desconexión. (c) State: se evaluó pensando en el crédito de SC-3, pero Cliente.cs no presenta hoy un comportamiento que cambie según un estado interno (bitácora B-06). (d) Elegida: Strategy, con una implementación de IDescuento por cada regla de descuento.
Qué sale y qué entra	IDescuento y ServicioDescuento no salen: se transforman. Pasan de ser una abstracción sin consumidor a ser la estrategia real que recibe ServicioVenta. Entran tres implementaciones nuevas: DescuentoSinDescuento , DescuentoPorcentaje y DescuentoMontoFijo , cada una con una regla de cálculo distinta bajo el mismo contrato.
Cómo se relaciona	Program.cs crea la estrategia inicial y la entrega por constructor a ServicioVenta, que la usa sin saber cuál implementación concreta está detrás. La estrategia se puede cambiar en cualquier momento con EstablecerEstrategiaDescuento () , método que también expone FarmaciaFachada. Este es el punto donde Strategy y Facade se conectan: la fachada solo delega el cambio de estrategia, no calcula el descuento ella misma.
Impacto	Creadas: DescuentoSinDescuento.cs, DescuentoPorcentaje.cs, DescuentoMontoFijo.cs. Modificadas: ServicioVenta.cs, FarmaciaFachada.cs, Program.cs. Eliminadas: ninguna. Si se autoriza SC-3, cada nueva regla comercial de descuento se resuelve creando una clase IDescuento adicional, sin tocar ServicioVenta ni la fachada.
Qué cuesta	Una clase nueva por cada regla de descuento. También se paga en indirección: para saber qué descuento está activo en un momento dado, hay que revisar qué estrategia fue inyectada o cambiada en tiempo de ejecución, en vez de leerlo directamente en ServicioVenta. Además, ServicioVenta guarda la estrategia en un campo mutable para permitir el cambio en caliente, lo que significa que dos ventas seguidas podrían usar reglas distintas si algo cambió la estrategia entre una y otra. Esto queda declarado como comportamiento intencional, no como un error.
Origen	Adoptado por el equipo con apoyo de IA. Según la bitácora B-04 , la idea de usar Strategy en lugar de Decorator fue propuesta por el equipo, y la IA la validó apoyándose en que IDescuento ya existía en el código. La decisión se respalda además con la bitácora B-09 , donde el equipo comprobó por su cuenta, buscando en el repositorio, que ServicioDescuento e IDescuento no tenían ningún uso real antes de aceptar esa afirmación de la IA.

ACTIVIDAD 4

Entregable 4.1.: matriz de verificación.

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Facade (FarmaciaFachada + ServicioVenta)	Refuerza	Tensionado pero compensado	Neutro	Tensionado pero compensado	Tensionado pero compensado
Factory Method (FabricaProducto + subclases)	Refuerza	Refuerza	Refuerza	Refuerza	Tensionado pero compensado
Strategy (IDescuento + subclases)	Refuerza	Refuerza	Refuerza	Refuerza	Refuerza

Facade

SRP: la fachada solo delega, no calcula. Todo el cálculo de venta vive en ServicioVenta.

OCP: cumple parcialmente porque si después agregamos una operación nueva (ejm: cancelar venta), hay que tocar FarmaciaFachada para exponerla, aunque la lógica interna quede en un servicio aparte. Pero eso es lo normal en una fachada, protege contra cambios internos, no contra operaciones nuevas.

LSP: no aplica, porque no hay subclases de la fachada.

ISP: cumple parcialmente porque no existe IFarmaciaFachada en Program.cs conoce la clase concreta con todos sus métodos, aunque cada pantalla del menú se use solo uno. No es una interfaz grande, que este forzando algo, pero tampoco hay segregación.

DIP: cumple parcialmente porque el constructor inyecta bien (no usa new adentro), pero inyecta clases concretas (ServicioProducto, ServicioCliente...) porque no hay interfaces para esos servicios, es decir, cumple a medias, bien en los repositorios, pero no tanto en la capa de servicios.

Factory

SRP: cada fabrica sabe construir un solo tipo de producto, nada más.

OCP: el caso más limpio del proyecto, porque una variante nueva es una subclase nueva, sin tocar nada existente.

LSP: todas las subclases cumplen el mismo contrato, y devuelven un producto valido.

ISP: FabricaProducto tiene un solo método abstracto.

DIP: RepositorioMedicamentoArchivo recibe la fábrica opcional, pero si no se inyecta, se crea su propia FabricaMedicamentoCapsula por defecto

Strategy

SRP: cada estrategia calcula un solo tipo de descuento, sin saber nada de ventas ni de stock.

OCP: una regla nueva es una clase nueva implementando IDescuento, sin tocar ServicioVenta.

LSP: se cumple que el descuento no deja el total en negativo.

ISP: IDescuento tiene un solo método, que es pequeño y enfocado.

DIP: ServicioVenta solo conoce la abstracción IDescuento, la implementación concreta la decide Program.cs y se inyecta por constructor.

Entregable 4.2.: Evidencias de que el comportamiento no cambia.

ID	Funcionalidad	Resultado AS-IS (Fase 1)	Resultado TO-BE (Fase 6)	¿Coincide?
CP-01	Carga de productos	Carga los 10 medicamentos	Carga los mismos 10 + 6 de SC-1 = 16 productos, sin error	Sí
CP-02	Listado de productos	Muestra nombre/stock/precio	Mismo formato para los 10 originales (Dolex: stock 2, \$5.000)	Sí
CP-03	Búsqueda existente	No hace falta escribir el nombre completo ni respetar mayúsculas para encontrar el producto	Se escribió "dole" (incompleto y en minúscula) y encontró "Dolex" igual que antes	Sí
CP-04	Búsqueda sin coincidencias	Si el nombre no existe, el sistema avisa que no lo encontró, sin sugerir algo parecido	Con un nombre inventado que no existe, dice exactamente lo mismo: no lo encontró, sin inventar una respuesta parecida	Sí
CP-05	Registro de venta	Descuenta stock, registra movimiento	Igual (Omeprazol 20 -> 17)	Sí
CP-06	Venta en límite de stock	Permite vender todo el disponible	Igual (Dolex 2 -> 0)	Sí
CP-07	Alerta de stock mínimo	Se dispara si stock <= mínimo	Igual (Amoxicilina, 1 <= 5)	Sí

CP-08	Alerta de vencimiento	Se dispara si faltan <= 30 días	Mismo criterio	Sí
CP-09	Acumulación de puntos	Los puntos nuevos se suman a los que ya tenía el cliente; no hace falta escribir su nombre completo ni en mayúsculas para encontrarlo	Se escribió "carlo" (incompleto y en minúscula) y encontró a "Carlos", sumándole los 30 puntos igual que antes	Sí
CP-10	Autenticación	Validas entran, invalidas se niegan	Igual en ambos casos	Sí
CN-01	Venta de servicio clínico (SC-2)	No existía en Reto 1	Se vende "Inyectología" como IVendible, descuento 10% igual que un producto Cambiada a 15% en caliente, sí se aplica a la venta siguiente	
CN-02	Estrategia de descuento	ServicioDescuento existía, pero con 0 usos (P-05)	Cambiada a 15% en caliente, sí se aplica a la venta siguiente	
CN-03	Extensibilidad OCP (Factory Method)	Agregar tipo obligaba a editar el método central	Producto nuevo construido sin tocar RepositorioMedicamentoArchivo	
CN-04	Venta con stock insuficiente	Se rechaza limpio, sin excepción	Igual, tras extraer ServicioVenta de la fachada	

CP: caso de prueba, son los 10 casos que se identificaron en el reto 1, sirven para comprobar que el comportamiento no cambio con los patrones nuevos.

CN: caso nuevo, son los 4 casos que agregamos, se prueba los patrones implementados (Fachada, Factory, Strategy) y la funcionalidad de SC-2.

CP-03 — Búsqueda existente

<pre>===== SISTEMA FARMACIA ===== 1. Ver productos 2. Ver clientes 3. Buscar producto 4. Registrar venta 5. Acumular puntos 6. Ver alertas 7. Salir 8. Ver servicios clínicos (SC-02) Seleccione opción: 3 Ingrese nombre producto: dole Producto: Dolex Precio: 5000 Stock: 2</pre>	<pre>===== SISTEMA FARMACIA ===== 1. Ver productos 2. Ver clientes 3. Buscar producto 4. Registrar venta 5. Acumular puntos 6. Ver alertas 7. Salir Seleccione opción: 3 Ingrese nombre producto: dole Producto: Dolex Precio: 5000 Stock: -8</pre>
---	--

Se buscó el producto escribiendo "dole" (incompleto y en minúscula). En ambas versiones el sistema encontró "Dolex" igual, confirmando que la búsqueda parcial e insensible a mayúsculas se conserva sin cambios.

CP-04 — Búsqueda sin coincidencias

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir
8. Ver servicios clínicos (SC-02)

Seleccione opción: 3

Ingrese nombre producto: Gaviscon

Producto no encontrado
```

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: 3

Ingrese nombre producto: Gaviscon

Producto no encontrado
```

Se buscó un nombre inventado que no existe en el catálogo. En ambas versiones el sistema respondió que el producto no fue encontrado, sin sugerir ninguna coincidencia aproximada, igual que en el comportamiento original.

CP-05 — Registro de venta

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir
8. Ver servicios clínicos (SC-02)

Seleccione opción: 4

Nombre producto: Omeprazol
Cantidad: 3
Movimiento registrado: Venta

Venta registrada
Subtotal: 45.000 | Descuento: 4.500 | Total a pagar: 40.500

=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir
8. Ver servicios clínicos (SC-02)

Seleccione opción: 1
```

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: 4

Nombre producto: Omeprazol
Cantidad: 3
Movimiento registrado: Venta

Venta registrada

=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: 1
```

```
===== PRODUCTOS =====
```

Nombre	Stock	Precio
-----	-----	-----
Dolex	2	5000
Ibuprofeno	10	7000
Amoxicilina	1	12000
Acetaminofen	8	4500
Omeprazol	17	15000
Loratadina	4	9000
VitaminaC	15	6000
Diclofenaco	3	11000
Aspirina	9	7500
Naproxeno	7	13500
CremaNivea	20	15000
ShampooHead	15	12000
ProtectorSolar	8	25000
CocaCola	50	3500
Agua	100	1500
Snickers	30	2500

```
===== PRODUCTOS =====
```

Nombre	Stock	Precio
-----	-----	-----
Dolex	2	5000
Ibuprofeno	10	7000
Amoxicilina	1	12000
Acetaminofen	8	4500
Omeprazol	17	15000
Loratadina	4	9000
VitaminaC	15	6000
Diclofenaco	3	11000
Aspirina	9	7500
Naproxeno	7	13500
CremaNivea	20	15000
ShampooHead	15	12000
ProtectorSolar	8	25000
CocaCola	50	3500
Agua	100	1500
Snickers	30	2500

Se vendieron 3 unidades de Omeprazol en ambas versiones. El stock pasó de 20 a 17 en los dos casos, y el movimiento se registró de forma idéntica.

CP-07 — Alerta de stock mínimo

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir
8. Ver servicios clínicos (SC-02)

Seleccione opción: 6

Verificando alertas...
ALERTA: stock mínimo de Dolex
ALERTA: stock mínimo de Amoxicilina
ALERTA: stock mínimo de Loratadina
ALERTA: stock mínimo de Diclofenaco
ALERTA: Dolex próximo a vencer
ALERTA: Ibuprofeno próximo a vencer
ALERTA: Amoxicilina próximo a vencer
ALERTA: Acetaminofen próximo a vencer
ALERTA: Omeprazol próximo a vencer
ALERTA: Loratadina próximo a vencer
ALERTA: VitaminaC próximo a vencer
ALERTA: Diclofenaco próximo a vencer
ALERTA: Aspirina próximo a vencer
ALERTA: Naproxeno próximo a vencer
ALERTA: ShampooHead próximo a vencer
ALERTA: ProtectorSolar próximo a vencer
ALERTA: CocaCola próximo a vencer
ALERTA: Snickers próximo a vencer
```

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: 6

Verificando alertas...
ALERTA: stock mínimo de Dolex
ALERTA: stock mínimo de Amoxicilina
ALERTA: stock mínimo de Loratadina
ALERTA: stock mínimo de Diclofenaco
ALERTA: Dolex próximo a vencer
ALERTA: Ibuprofeno próximo a vencer
ALERTA: Amoxicilina próximo a vencer
ALERTA: Acetaminofen próximo a vencer
ALERTA: Omeprazol próximo a vencer
ALERTA: Loratadina próximo a vencer
ALERTA: VitaminaC próximo a vencer
ALERTA: Diclofenaco próximo a vencer
ALERTA: Aspirina próximo a vencer
ALERTA: Naproxeno próximo a vencer
ALERTA: ShampooHead próximo a vencer
ALERTA: ProtectorSolar próximo a vencer
ALERTA: CocaCola próximo a vencer
ALERTA: Snickers próximo a vencer
```

Se ejecutó la verificación de alertas con Amoxicilina en stock igual o menor a su mínimo configurado. En ambas versiones se generó la misma alerta, con el mismo mensaje y color.

CN-01 — Venta de servicio clínico

```
===== SERVICIOS CLÍNICOS (SC-02) =====
Nombre      Precio  Descripción
-----
Inyectología      15000  Aplicacion intramuscular o intravenosa
Cambio de vendajes 12000  Curacion y colocacion de vendajes esteriles
Curaciones basicas 20000  Limpieza, desinfeccion y curacion de heridas menores

=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir
8. Ver servicios clínicos (SC-02)

Seleccione opción: 4

Nombre producto: Inyectologia
Cantidad: 2
Movimiento registrado: Venta

Venta registrada
Subtotal: 30.000 | Descuento: 3.000 | Total a pagar: 27.000
```

Se registró la venta de "Inyectología" desde la opción 8 del menú, una funcionalidad que no existía en el sistema original. La venta se completó correctamente como un ítem vendible más, sin generar ningún error.

CN-04 — Venta con stock insuficiente

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir
8. Ver servicios clínicos (SC-02)

Seleccione opción: 4

Nombre producto: Dolex
Cantidad: 10

Stock insuficiente. Disponible: 2
```

```
=====
SISTEMA FARMACIA
=====
1. Ver productos
2. Ver clientes
3. Buscar producto
4. Registrar venta
5. Acumular puntos
6. Ver alertas
7. Salir

Seleccione opción: 4

Nombre producto: Dolex
Cantidad: 10
Movimiento registrado: Venta

Venta registrada
```

Se intentó vender una cantidad mayor a la disponible en stock. En ambas versiones el sistema respondió de la misma forma, confirmando que el comportamiento se mantiene igual tras separar ServicioVenta de la fachada.

ACTIVIDAD 5

Entregable 5.1.: registro de riesgos.

ID	Riesgo (si ocurre X, entonces Y)	Prob	Imp	Exp	Qué hacen para evitarlo	Cómo se enteran de que está pasando (el riesgo se está materializando)
R-01	Si FarmaciaFachada empieza a incluir lógica de negocio y no solo coordinación, vuelve a concentrar responsabilidades como antes.	3	4	12	Según lo propuesto, el punto de entrada se limitará a recibir los componentes ya construidos y a delegar cada acción; los cálculos y reglas de negocio se mantendrán en los servicios especializados.	Si alguien necesita entender una regla de negocio y tiene que buscarla en el punto de entrada en vez del servicio correspondiente, es señal de que está absorbiendo responsabilidades de más.
R-02	Si IDescuento queda con un descuento activo por defecto, el total de venta cambia frente al original sin autorización.	4	5	20	Según lo propuesto, el mecanismo de descuentos se diseñará para poder intercambiarse fácilmente, incluyendo la opción de no aplicar ningún descuento, buscando mantener el comportamiento original salvo que el negocio indique lo contrario.	Al comparar el total de una venta antes y después del cambio, con los mismos datos, el valor no coincide sin que exista una solicitud de negocio que lo respalde.
R-03	Si las fábricas de producto no tienen un punto claro que decida cuál usar, agregar un tipo nuevo sigue exigiendo tocar el arranque del sistema.	3	3	9	Según lo propuesto, la creación de cada tipo de producto se aislará en un componente independiente, de forma que un tipo nuevo se sume agregando un componente adicional en vez de modificar uno existente.	Al poner en marcha un tipo de producto nuevo, se necesita modificar el arranque del sistema (y no solo añadir el componente nuevo) para que empiece a usarse.
R-04	Si los repositorios de producto repiten su propia lógica de lectura de archivo, un ajuste de formato exige tocar varios lugares.	3	2	6	Según lo propuesto, se separará qué se crea de cómo se obtienen los datos, de modo que cada componente de lectura dependa de una pieza de creación intercambiable en vez de tener la lógica de creación mezclada.	Al corregir un error de formato en los datos de entrada, la corrección debe replicarse en más de un componente para que el efecto sea consistente.

ACTIVIDAD 6

Ambas vistas están presentes en el repositorio ubicado en el GitHub para una mejor calidad y formato de visualización.